

onnx-light

onnx protobuf free

C++ only
parallelized

...

Xavier Dupré

Protobuf...

Since I started my journey on ONNX, protobuf has always been a pain:

- Frequent conflicts between the installed version and the one used by ONNX
- Unable to switch from Python to C++ without serialization
- Complex compilation since we need to generate a header first.
- Unable to extend the API.
- 2 Gb limitation

Let's try without.

First question?

Can we do protobuf without protobuf?

[ONNX2: prototype for a replacement of protobuf = same API, same format as ONNX but without protobuf by xadupre · Pull Request #7208 · onnx/onnx](#)

So it is possible.

What's next? Smooth rewriting or complete rewriting from scratch?

Maybe two ONNX are better: users can implement a switch and check both at the same time rather than introducing major breaking changes. **Let's rewrite.**

No protobuf but also

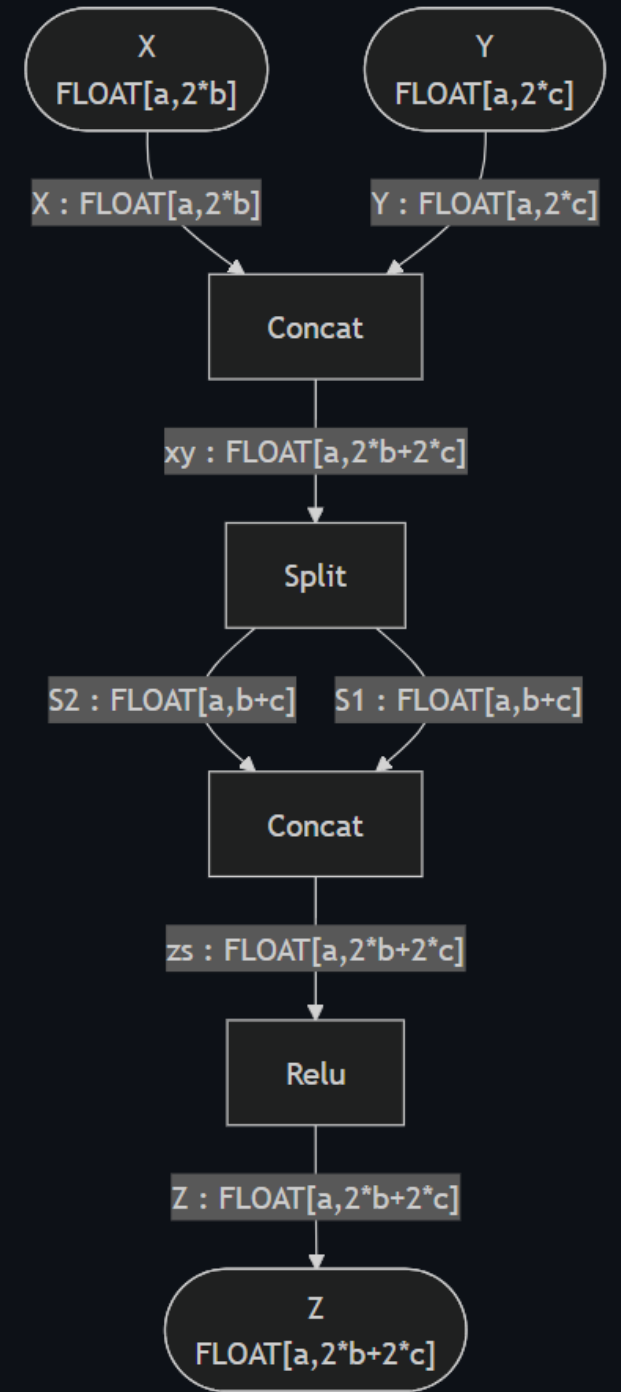
- No protobuf
 - removes the 2 GB limitation
 - Parallelization
 - Implement custom loading strategies (read only part of a model without loading the entire file, etc.), alignment
 - Native C++ support for weight files
 - Minimize dependencies and security issues
- No serialization layer between Python and C++.
- Same C++ and Python APIs `import onnx_light.onnx as onnx`
- No auto-generated Markdown files in the repository: no longer PR with 200 files modified.

Other requirements (or things I'd like to change)

- No backend tests stored as files: backend tests implemented as C++ functions.
- Backend tests are generated from a shared random test suite
- Expected outputs are produced by C++ kernels.
- Kernels can be parallelized except reductions to ensure deterministic results.
- Flexible C++ linking: users can link only the reading/writing/serialization components, or include schemas, kernels, and/or backend tests as needed.

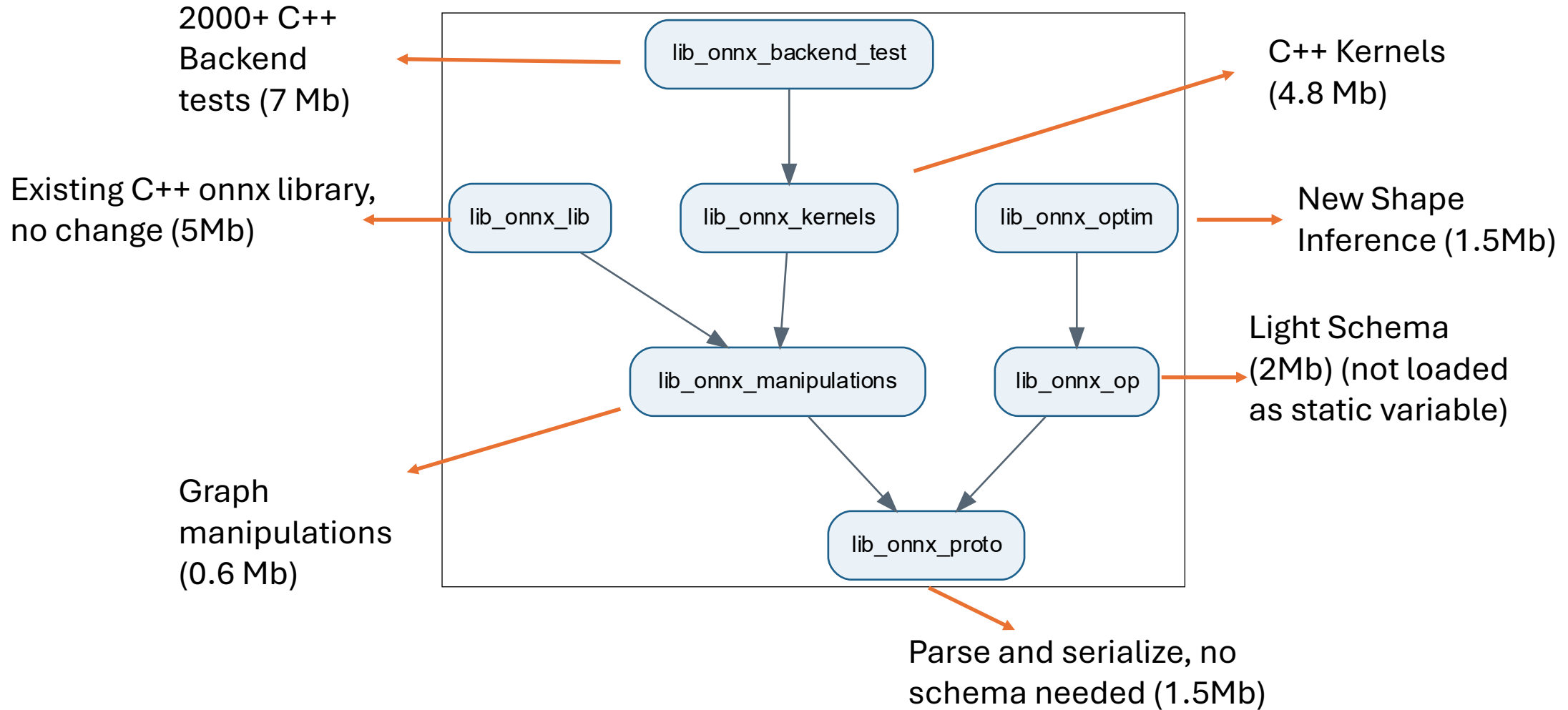
And Symbolic Shape Inference

We need symbolic shapes instead of
unk__0, unk__1...



What did I get?

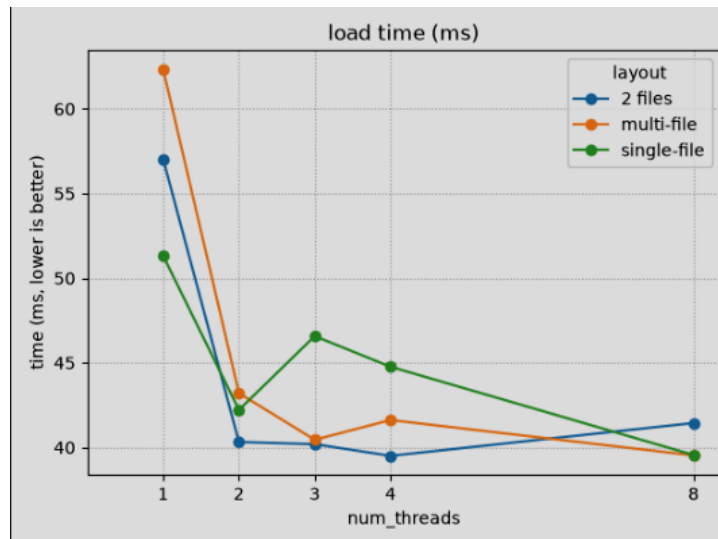
Core pieces: take what you need



Parsing and Serializing Options

- Data can be aligned when loading tensor
- Parallelization is possible.
- Onnxruntime flatbuffers is supported
- More secure: limit recursivity

Other scenarios can be implemented such a callback to allocate directly on GPU or any other memory.



```
int64_t raw_data_threshold = kSmallTensorDataThresholdBytes;
int64_t alignment = 0;
SerializeFormat format = SerializeFormat::kOnnx;
bool skip_raw_data = false;
int32_t num_threads = 1;
int64_t min_parallel_block_size = 0;
bool no_copy = false;
FileLoadMode file_load_mode = FileLoadMode::kAuto;
int32_t max_recursion_depth = 50;
int64_t max_tensor_size_bytes = 0;
```

```
int64_t raw_data_threshold = kSmallTensorDataThresholdBytes;
int64_t alignment = 0;
SerializeFormat format = SerializeFormat::kOnnx;
int32_t num_threads = 1;
int64_t min_parallel_block_size = 0;
bool use_external_data_location = true;
int64_t max_external_file_size = 0;
```

A kernel

```
Tensor Not::operator()(const Tensor &x) const {
    Tensor y("", DataType::BOOL, x.shape,
        std::vector<uint8_t>(static_cast<size_t>(x.element_count())));
    (*this)(x, y);
    return y;
}

void Not::operator()(const Tensor &x, Tensor &output) const {
    EXT_ENFORCE_INVALID(x.data_type == DataType::BOOL, "kernel::Not only supports BOOL tensors.");
    EXT_ENFORCE_INVALID(output.data_type == DataType::BOOL,
        "kernel::Not preallocated output must be a BOOL tensor.");
    EXT_ENFORCE_INVALID(output.shape == x.shape,
        "kernel::Not preallocated output shape must match input shape.");
    const int64_t n = x.element_count();
    const size_t expected_bytes = static_cast<size_t>(n);
    EXT_ENFORCE_INVALID(output.data.size() == expected_bytes,
        "kernel::Not preallocated output buffer has unexpected size in bytes.");
    const uint8_t *px = x.bytes();
    uint8_t *py = output.data.data();
    for (int64_t i = 0; i < n; ++i) {
        py[static_cast<size_t>(i)] = static_cast<uint8_t>(px[i] == 0 ? 1 : 0);
    }
}
```

Simple kernel

```
void Dispatch(const Tensor &x, float alpha, Tensor &output) {
    switch (static_cast<DataType>(x.data_type)) {
    case DataType::FLOAT:
        ComputeInPlace<float>(x, alpha, output);
        return;
    case DataType::DOUBLE:
        ComputeInPlace<double>(x, static_cast<double>(alpha), output);
        return;
    case DataType::FLOAT16:
        ComputeHalf(x, alpha, output, Float16BitsToFloat, FloatToFloat16Bits);
        return;
    case DataType::BFLOAT16:
        ComputeHalf(x, alpha, output, Bfloat16BitsToFloat, FloatToBfloat16Bits);
        return;
    default:
        EXT_THROW_INVALID(kName, ": unsupported data type ", x.data_type,
            ", only supports FLOAT, DOUBLE, FLOAT16, and BFLOAT16 tensors.");
    }
}
```

Dispatch mechanism to support all types.

Backend test

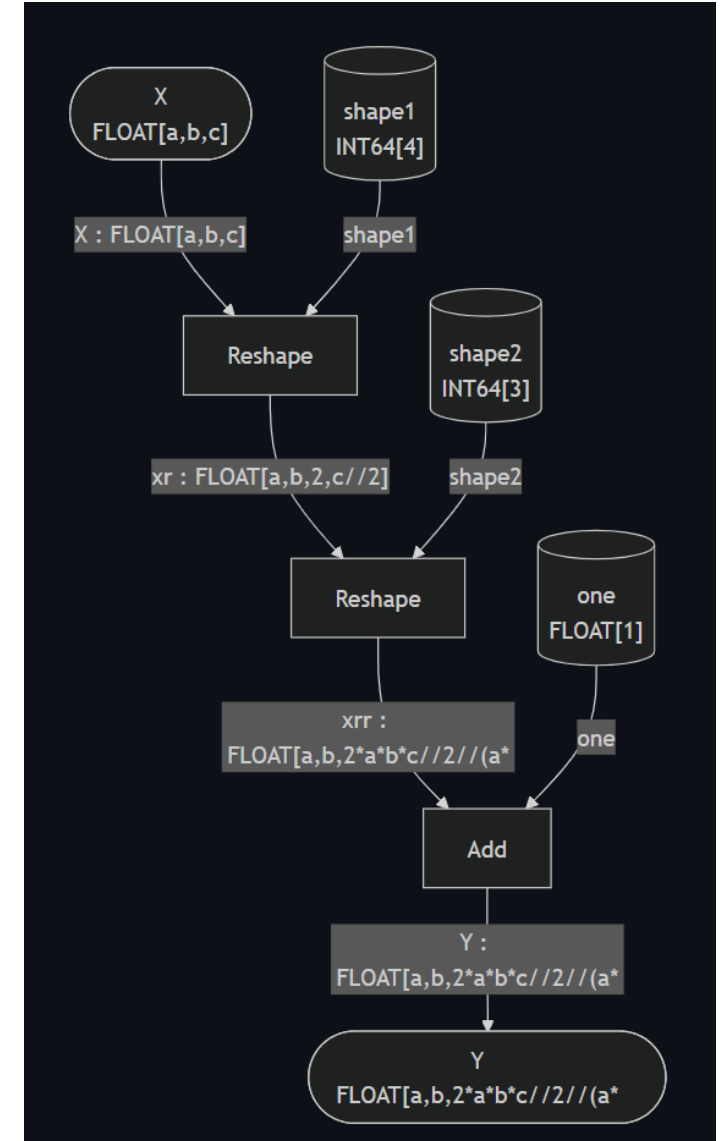
Very similar to the existing ones except it does not write any file.

```
void RegisterWhereCases(std::vector<TestCase> &registry) {  
    const OpsetId opset = DefaultOpset(16);  
    const kernel::KernelContext ctx{opset};  
    const kernel::Where where_kernel{ctx};  
  
    {  
        NodeProto node = MakeNode("Where", {"condition", "x", "y"}, {"output"});  
  
        Tensor condition = Tensor::FromBool("condition", {2, 2}, {1, 0, 1, 0});  
        Tensor x = Tensor::FromFloat("x", {2, 2}, {1.0f, 2.0f, 3.0f, 4.0f});  
        Tensor y = Tensor::FromFloat("y", {2, 2}, {5.0f, 6.0f, 7.0f, 8.0f});  
        Tensor output = where_kernel(condition, x, y);  
  
        Expect(node, {condition, x, y}, {output}, "test_where_example", {opset}, "backend-test",  
            registry);  
    }  
  
    {  
        NodeProto node = MakeNode("Where", {"condition", "x", "y"}, {"output"});  
  
        Tensor condition = Tensor::FromBool("condition", {2, 1}, {1, 0});  
        Tensor x = Tensor::FromInt32("x", {2, 3}, {1, 2, 3, 4, 5, 6});  
        Tensor y = Tensor::FromInt32("y", {1, 3}, {10, 20, 30});  
        Tensor output = where_kernel(condition, x, y);  
  
        Expect(node, {condition, x, y}, {output}, "test_where_bcast", {opset}, "backend-test",  
            registry);  
    }  
}
```

New tests related to shape inference

value_info is filled with expected values.

intermediate	op type	kind	expected	onnx	onnx-light-optim
X	—	input	FLOAT (H, 2*h)	—	—
resized_output	Resize	value_info	FLOAT (H//2, h)	X FLOAT (unk__0, unk__1)	✓ FLOAT (H//2, h)
tile_out	Tile	value_info	FLOAT (2*(H//2), 2*h)	X FLOAT (unk__2, unk__3)	✓ FLOAT (2*(H//2), 2*h)
output	Max	output	FLOAT (2*(H//2), 2*h)	X FLOAT (unk__2, unk__3)	✓ FLOAT (2*(H//2), 2*h)



Symbolic Expressions

- Only symbols, no function to keep short expressions
- 2 divisions: // and /:
 - // to round the results
 - /: when the exact result is an integer

When `Reshape(x[N], [-1, 2])`, we know that `N` is even so we use `N/:2` and not `N//2`.

`2*(N/:2) == N` but `2*(N//2)` may be different.

```
EXPECT_EQ(get_str(simplify_expression("2*x//2")), "x");
EXPECT_EQ(get_str(simplify_expression("(2*x)//2")), "x");
EXPECT_EQ(get_str(simplify_expression("(x*y)//y")), "x");
EXPECT_EQ(get_str(simplify_expression("(x*(y+1))/(y+1)")), "x");
EXPECT_EQ(get_str(simplify_expression("((c)/(2))")), "c//2");
```

Operator	Meaning
<code>^</code>	Encodes <code>max(a, b)</code>
<code>&</code>	Encodes <code>min(a, b)</code>
<code>+</code>	Addition
<code>-</code>	Subtraction
<code>*</code>	Multiplication
<code>//</code>	Floor (integer) division
<code>/:</code>	Exact (integer) division — see below
<code>%</code>	Modulo
Unary	Unary <code>-</code> / <code>+</code>

Next steps



- Implements a switch in ir-py to check it is compatible with `onnx_light.onnx`
- Do the same for onnxruntime and validate the C++ API
- Supports more loading/saving scenarios
- Expose standard graph transformations (dead end removal, ...)

Source and Documentation

- Source: [xadupre/onnx-light: onnx without protobuf](#)
- Documentation: [onnx-light — onnx-light 0.1.1 documentation](#)
- Evaluation of onnx-light on onnx backend tests: [onnx — backend node test coverage](#)
- All backend tests in onnx-light: [onnx-light — backend test coverage \(onnxruntime vs reference vs onnx-light\)](#)
- Comparison of shape inference algorithms: [onnx-light — shape inference coverage \(onnx-light vs onnx-light.onnx_optim vs onnx vs onnx-shape-inference vs onnxruntime.transformers vs yobx\)](#)